

МИССИЯ: ХАКЕРСКИЙ ТЕРМИНАЛ

МОДУЛЬ: ОСНОВЫ КОМПОНОВКИ (UI) | ТЕОРИЯ

ЧТО ТАКОЕ ANDROID-РАЗРАБОТКА?

Android - это операционная система для смартфонов и планшетов, которую использует большинство людей по всему миру. Когда ты открываешь приложение на телефоне, за его работой стоит код, написанный программистом. Сегодня мы начнём создавать свои собственные приложения!

Для создания приложений Android разработчики используют специальные языки программирования. Мы будем использовать Kotlin - современный и мощный язык, созданный компанией JetBrains (теми же, кто сделал IntelliJ IDEA).

Главная программа, в которой мы будем работать - это Android Studio. Это специальная среда разработки, которая помогает писать код, проверять ошибки и видеть результат прямо на экране.

ЧТО ТАКОЕ JETPACK COMPOSE?

Jetpack Compose - это новый способ создания интерфейса для приложений Android. Раньше программисты описывали внешний вид приложения в специальных XML-файлах, которые были похожи на HTML-страницы. Это было долго и неудобно.

С Compose мы пишем интерфейс прямо на Kotlin! Каждая кнопка, текст или картинка - это вызов функции. Представь, что ты строишь конструктор LEGO: каждая функция - это кирпичик, а из этих кирпичиков мы собираем целый экран приложения.

Compose автоматически следит за изменениями и перерисовывает только те части экрана, которые изменились. Это делает наши приложения быстрыми и отзывчивыми.

ЧТО ТАКОЕ КОМПОЗИБЛ ФУНКЦИЯ?

@Composable - это специальная пометка (аннотация) перед функцией. Она говорит Android: 'Эта функция не просто считает числа, она рисует элементы на экране'.

Любая функция с @Composable может вызывать другие функции с @Composable внутри себя. Так мы строим иерархию: большой экран содержит колонку, колонка содержит текст, текст содержит буквы.

Каждый элемент Compose называется 'композиблом' (composable). Когда композибл меняется - Android автоматически обновляет только его на экране.

МИССИЯ: ХАКЕРСКИЙ ТЕРМИНАЛ

МОДУЛЬ: ОСНОВЫ КОМПОНОВКИ (UI) | ТЕОРИЯ

КАК СОЗДАТЬ ПЕРВЫЙ ЭКРАН В ANDROID STUDIO

Открой Android Studio и создай новый проект: File -> New -> New Project. Выбери шаблон 'Empty Activity' с поддержкой Compose.

В панели слева (Project) найди файл MainActivity.kt внутри папки app/src/main/java/твой_пакет/. Это главный файл, где мы будем писать код нашего приложения.

Открой этот файл и удали весь существующий код. Мы напишем всё с нуля!

Теперь создай новую функцию экрана. Напиши @Composable сверху, затем fun ИмяЭкрана() { }. Всё содержимое экрана пишется внутри фигурных скобок.

Чтобы увидеть результат без запуска на телефон, используй аннотацию @Preview над отдельной функцией. Android Studio покажет предпросмотр прямо в редакторе.

МИССИЯ: ХАКЕРСКИЙ ТЕРМИНАЛ

МОДУЛЬ: ОСНОВЫ КОМПОНОВКИ (UI) | ID: 1

ЭТАП 1: 0. Создаём функцию экрана (ОБЯЗАТЕЛЬНО)

```
// Android Studio подскажет нужные импорты (нажми Alt+Enter на красном тексте)

// Создаём нашу функцию-экран:
@Composable
fun HackerAccessScreen() {
    // ВСЁ СОДЕРЖИМОЕ ИЗ СЛЕДУЮЩИХ ШАГОВ ПИШЕМ ВНУТРИ ЭТОЙ ФУНКЦИИ!
    // Следи за отступами - всё должно быть внутри фигурных скобок {}
```

ЭТАП 2: 1. Создаем 'Черную дыру' (Фон)

```
// Box это коробка, которая накладывает элементы друг на друга.
// Мы используем её, чтобы отцентровать наш терминал.
Box(
    modifier = Modifier
        .fillMaxSize()
        .background(Color(0xFF050505)),
    contentAlignment = Alignment.Center
) { /* Внутри пишем код из следующего шага */ }
```

ЭТАП 3: 2. Главная рамка: Секрет Двойного Отступа

```
// ВАЖНО: Первый padding это отступ СНАРУЖИ рамки (Margin).
// Второй padding это отступ ВНУТРИ рамки (Padding).
Column(
    modifier = Modifier
        .padding(24.dp) // Внешний зазор
        .fillMaxWidth()
        .border(1.dp, Color(0xFF00F3FF).copy(0.5f), RoundedCornerShape(8.dp))
        .padding(20.dp), // Внутренний зазор
    horizontalAlignment = Alignment.CenterHorizontally
) { /* Тут будет содержимое терминала */ }
```


МИССИЯ: ХАКЕРСКИЙ ТЕРМИНАЛ

МОДУЛЬ: ОСНОВЫ КОМПОНОВКИ (UI) | ID: 1

ЭТАП 4: 3. Заголовок системы (Иконка + Текст)

```
// Row ставит элементы в ряд. Spacer создает пустое пространство.
Row(verticalAlignment = Alignment.CenterVertically) {
    Icon(
        imageVector = Icons.Default.Terminal,
        contentDescription = null,
        tint = Color(0xFF00F3FF),
        modifier = Modifier.size(24.dp)
    )
    Spacer(Modifier.width(12.dp))
    Text(
        text = "СИСТЕМА ВЗЛОМА v1.0",
        color = Color(0xFF00F3FF),
        fontWeight = FontWeight.Bold,
        letterSpacing = 2.sp
    )
}
```

ЭТАП 5: 4. Красная тревога и Кнопка взлома

```
Spacer(Modifier.height(32.dp))

Text(
    text = "ВНИМАНИЕ: ОБНАРУЖЕНО ВТОРЖЕНИЕ",
    color = Color.Red,
    fontSize = 12.sp,
    fontWeight = FontWeight.Black
)

Spacer(Modifier.height(16.dp))

Button(
    onClick = { },
    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF00F3FF)),
    shape = RoundedCornerShape(4.dp),
    modifier = Modifier.fillMaxWidth()
) {
    Text("ПОДКЛЮЧИТЬСЯ", color = Color.Black, fontWeight = FontWeight.ExtraBold)
}
```


МИССИЯ: ХАКЕРСКИЙ ТЕРМИНАЛ

МОДУЛЬ: ОСНОВЫ КОМПОНОВКИ (UI) | ID: 1

ЭТАП 6: 5. Финальные штрихи (Инфо-панель)

```
Spacer(Modifier.height(12.dp))

// Текст с прозрачностью (alpha) выглядит более технологично
Text(
    text = "IP: 192.168.1.104 | ПИНГ: 24мс",
    color = Color(0xFF00F3FF).copy(alpha = 0.4f),
    fontSize = 10.sp
)
```

ЭТАП 7: 6. Как посмотреть результат (Preview)

```
// Добавь это ВНЕ основной функции HackerAccessScreen(), чтобы увидеть экран
@Preview
@Composable
fun HackerAccessScreenPreview() {
    MaterialTheme {
        HackerAccessScreen()
    }
}

// В Android Studio справа появится панель Preview с предпросмотром!
// Если не появляется - нажми Build -> Make Project
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Измени цвет текста на `Color.Green`, измени ПИНГ на 10мс и добавь еще одну иконку `Icons.Default.Lock` в заголовок.